

Cross-Architecture Interpretability Audit: Neural CDE vs Mamba2 (T4-safe, apples-to-apples)

0) Why the run is getting killed (*it's not Hessian*)

The new, rather lengthy, comparison script currently does:

- **Loads the entire 5.8 GB CSV into a pandas DataFrame** via
`pd.read_csv(data_path)`

`audit_cde_comparison`

That alone can exceed RAM (pandas often expands memory well beyond file size due to parsing + object overhead + copies). The log shows the process was **killed right after starting CSV read**, before model comparison begins.

Separately, even if we fix CSV loading, the script has two **silent GPU/CPU memory bombs**:

1. **Gradient saliency stores raw gradients for every sampled window** (`raw_grads = np.stack(all_grads)`)

MULTI-MODEL NEURAL CDE INTERPRE...

, which is avoidable.

2. **Fisher diagonal allocates a full `zeros_like(param)` tensor for every parameter**

MULTI-MODEL NEURAL CDE INTERPRE...

. For large models, that duplicates the model's parameter footprint in memory (often on GPU), which is catastrophic.

So the fix is: **stream the dataset + replace “store everything” analytics with streaming/accumulating estimators.**

1) Architectural differences that matter for interpretability

1.1 Neural CDE (continuous-time, vector-field semantics)

A Neural CDE can be written:

$$Z_T = Z_0 + \int_0^T f_\theta(Z_t) dX_t$$

- $X_t \in \mathbb{R}^D$: the (interpolated) input path (features over time).
- $Z_t \in \mathbb{R}^H$: latent state.
- $f_\theta: \mathbb{R}^H \rightarrow \mathbb{R}^{H \times D}$: learned **vector field**.
- The model's "reasoning" often corresponds to **geometric sensitivity of Z_t to increments dX_t** .

A discrete Euler form (useful for interpretability intuition):

$$Z_{t+1} \approx Z_t + f_\theta(Z_t) (X_{t+1} - X_t)$$

- Attribution has a clean decomposition: effects come from **input increments** and **vector field**.

Interpretability consequence: gradients/saliency are influenced by (i) the solver/interpolation, and (ii) the learned vector field. Permutation importance is fairly stable, but **gradient metrics must be computed carefully** (autocast + batch control).

1.2 Mamba2 (discrete selective SSM + gating, long-range via state update)

Mamba2-style layers are **state-space models** with selective (input-dependent) parameterization. A simplified form:

$$h_{t+1} = A(x_t) h_t + B(x_t) x_t, y_t = C(x_t) h_t$$

- $x_t \in \mathbb{R}^D$: timestep input.
- $h_t \in \mathbb{R}^H$: SSM hidden state.
- $A(\cdot), B(\cdot), C(\cdot)$: **input-dependent** ("selective") maps.
- Internally, implementations use fused scans / causal conv; attribution can concentrate in **gating paths**.

Interpretability consequence:

- Gradient saliency is often **sharper and more localized** (gate-driven) than CDE.
- Hessian/Fisher are more likely to explode memory because of parameter counts and fused kernels.

1.3 What “fair” means across these two

A fair audit must make the two models comparable on:

1. **Same input contract:** same features, scaling, NaN policy, leakage mask, and **same window length T** (we already enforce T_{common} in spirit).
 2. **Same output contract:** same head mapping and ROI head selection.
 3. **Comparable attribution objects:**
 - For CDE: attribute to X_t or increments ΔX_t .
 - For Mamba2: attribute to x_t (and optionally gating activations).
 4. **Comparable inequality/predicate outputs:** if either model produces predicate logits/activations, those must be normalized into a common “predicate probability” space.
-

2) Inequality learning alignment (mathematical retrofitting)

The pipeline includes differentiable predicate discovery / inequality logic (see predicate discovery engine)

`predicate_discovery`

and training exports discovered predicates into checkpoints

`train_condor_brain`

.

2.1 Canonical predicate probability

Assume each model can produce predicate logits $\ell \in \mathbb{R}^K$ (or some inequality score). Convert to a comparable probability:

$$p = \sigma\left(\frac{\ell - \mu}{\tau}\right)$$

- $\sigma(u) = \frac{1}{1+e^{-u}}$: sigmoid.
- μ : calibration offset (e.g., median logit over samples).
- $\tau > 0$: temperature (controls sharpness).

Why: Different architectures often scale logits differently; the temperature normalization makes predicate activations comparable.

2.2 Predicate distribution divergence

For each predicate k , compute mean probability $\bar{p}_k$ over samples; normalize into a categorical distribution:

$$P_k = \frac{\bar{p}_k + \varepsilon}{\sum_{j=1}^K (\bar{p}_j + \varepsilon)}$$

Then compute Jensen–Shannon divergence:

$$\text{JSD}(P, Q) = \frac{1}{2} \text{KL}(P \parallel M) + \frac{1}{2} \text{KL}(Q \parallel M), M = \frac{P + Q}{2}$$

This is already consistent with the divergence math in the audit spec

MULTI-MODEL NEURAL CDE INTERPRE...

, but the **key retrofit** is: build P, Q from **temperature-calibrated predicate probabilities**, not raw logits.

2.3 Inequality satisfaction rate (architecture-agnostic)

If we can evaluate each learned inequality predicate as a boolean (or soft truth), define:

$$\text{SatRate} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}\{p^{(i)} \geq \alpha\}$$

- α : truth threshold (e.g., 0.5 or calibrated).
- Compare SatRate across CDE vs Mamba2 **on the same sample windows**.

3) Required audit-script modifications (design)

3.1 Replace “CDE-only loader” with architecture-aware loader

The current audit script uses `load_cde_model(...)` and instantiates `CondorBrain(... use_cde=True ...)`

MULTI-MODEL NEURAL CDE INTERPRE...

, which is wrong for Mamba2 checkpoints.

Fix: `load_model_any(ckpt)` must:

- Read `model_config / config` from checkpoint.
- Determine `use_cde` boolean.
- Instantiate `CondorBrain(use_cde=use_cde)` (or the correct underlying module).
- Wrap `forward()` into a **standard output dict**.

3.2 Standardize forward() outputs (ModelAdapter)

The `CondorBrain` forward can return tuples with extra items (features, diffusion, experts, etc.)

`condor_brain`

. For interpretability, we want **one standard structure**:

```
1. {  
2.   "y": Tensor[B, H],           # the 10 heads  
3.   "pred_logits": Tensor[B, K] or None, # inequality/predicate logits if available  
4.   "extras": dict                # optional: regimes, routing info, etc.  
5. }  
6.
```

3.3 Make every metric streaming/batched

Key principles for a T4:

- Never allocate tensors shaped like **(num_params)** squared (Hessian).
- Never keep **all per-sample grads** in RAM.
- Never duplicate the model's parameters into same-shape buffers (the Fisher currently does)

MULTI-MODEL NEURAL CDE INTERPRE...

.

4) Memory-efficient strategies that actually work on a T4

4.1 Stream the CSV into sampled windows (no full DataFrame)

Replace `df = pd.read_csv(...)`

`audit_cde_comparison`

with a streaming window sampler. we only need:

- feature columns (`float32`)
- a deterministic sample of start indices
- the resulting N_{samples} windows of shape (T, D)

We can do this in one pass with a rolling buffer.

4.2 Permutation importance without cloning the entire batch per feature

Naive approach (current pattern): clone `x_base` for each feature $\rightarrow$ huge transient allocations.

Better approach:

- Keep windows on CPU.
- For each feature j , shuffle only that feature across samples **in CPU memory**, then run the model in **micro-batches**.

4.3 Gradient saliency: online accumulation (no `raw_grads`)

Instead of storing `raw_grads`

MULTI-MODEL NEURAL CDE INTERPRE...

, accumulate:

$$S_j = \frac{1}{N} \sum_{i=1}^N \frac{1}{T} \sum_{t=1}^T \left| \frac{\partial y_{\text{ROI}}^{(i)}}{\partial x_{t,j}^{(i)}} \right|$$

Compute it streaming:

- for each sample: compute `grad` $\rightarrow$ add `abs(grad).mean(axis=0)` to running sum.

4.4 Fisher: layerwise scalar Fisher (no per-parameter tensors)

The current Fisher uses `zeros_like(param)` for each param

MULTI-MODEL NEURAL CDE INTERPRE...

. Replace with:

$$\mathcal{F}_\ell \approx \frac{1}{N} \sum_{i=1}^N \sum_{p \in \ell} \left(\frac{\partial \mathcal{L}^{(i)}}{\partial \theta_p} \right)^2$$

Store only one float per layer/module, not a tensor.

4.5 Optional: CPU-side Fisher/HVP (only if needed)

If GPU is tight, we can:

- move model to CPU for Fisher (slow but safe),
- or compute only for selected modules (heads, last block, predicate module).

5) Deliverable scripts (drop-in solution)

Below are **two scripts**:

1. `intelligence/model_adapter.py`
2. `intelligence/audit_cde_vs_mamba2_comparison.py` (streaming data, batched metrics, no OOM)

We can paste them directly into the repo under `/scripts` or `/intelligence`.

5.1 `intelligence/model_adapter.py`

```

1. # intelligence/model_adapter.py
2. import os
3. import torch
4. from dataclasses import dataclass
5. from typing import Any, Dict, Optional, Tuple
6.
7. from intelligence.condor_brain import CondorBrain
8. from intelligence.canonical_feature_registry import INPUT_DIM_V22
9.
10. @dataclass
11. class ModelInfo:
12.     name: str
13.     ckpt_path: str
14.     seq_len: int
15.     use_cde: bool
16.     d_model: int
17.     n_layers: int
18.     n_params: int
19.

```

```

20. class ModelAdapter:
21.     """
22.     Wraps CondorBrain checkpoints (CDE or Mamba2) into a standard interface:
23.     forward(x) -> {"y": Tensor[B,H], "pred_logits": Optional[Tensor[B,K]], "extras": dict}
24.     """
25.     def __init__(self, model: torch.nn.Module, info: ModelInfo, device: torch.device):
26.         self.model = model
27.         self.info = info
28.         self.device = device
29.
30.     @torch.no_grad()
31.     def predict(self, x: torch.Tensor) -> Dict[str, Any]:
32.         """
33.         x: (B, T, D)
34.         """
35.         out = self.model(x)
36.         pred_logits = None
37.         extras: Dict[str, Any] = {}
38.
39.         if isinstance(out, tuple):
40.             # Convention in the codebase: first element is outputs
41.             y = out[0]
42.             # Heuristic: if any later element is a tensor shaped (B, K) treat as predicate
logits
43.             # We can tighten this by exposing return_predicates=True in CondorBrain if we want.
44.             for item in out[1:]:
45.                 if torch.is_tensor(item) and item.ndim == 2 and item.shape[0] == y.shape[0]:
46.                     # candidate predicate logits
47.                     if item.shape[1] > 10: # avoid confusing with heads
48.                         pred_logits = item
49.                         break
50.             extras["tuple_len"] = len(out)
51.         else:
52.             y = out
53.
54.         return {"y": y, "pred_logits": pred_logits, "extras": extras}
55.
56.     def _infer_use_cde_from_state_dict(state_dict: Dict[str, torch.Tensor]) -> Optional[bool]:
57.         keys = list(state_dict.keys())
58.         # Heuristics: adjust based on the actual module names if needed.
59.         if any("cde" in k.lower() or "vector_field" in k.lower() for k in keys):
60.             return True
61.         if any("mamba" in k.lower() or "ssm" in k.lower() or "selective_scan" in k.lower() for k in
keys):
62.             return False
63.         return None
64.
65.     def load_model_any(ckpt_path: str, device: torch.device, input_dim: int = INPUT_DIM_V22) ->
Tuple[ModelAdapter, Dict[str, Any]]:
66.         ckpt = torch.load(ckpt_path, map_location=device, weights_only=False)
67.
68.         seq_len = int(ckpt.get("seq_len", 256))
69.         config = ckpt.get("model_config", ckpt.get("config", {})) or {}
70.
71.         d_model = int(config.get("d_model", 128))
72.         n_layers = int(config.get("n_layers", 2))
73.
74.         # Determine architecture
75.         use_cde = config.get("use_cde", None)
76.         if use_cde is None:
77.             use_cde = config.get("cde", None)
78.         if use_cde is None:
79.             # infer from weights
80.             state_dict = ckpt.get("state_dict", ckpt)
81.             if any(k.startswith("module.") for k in state_dict.keys()):

```



```
82.         state_dict = {k.replace("module.", ""): v for k, v in state_dict.items()}
83.         inferred = _infer_use_cde_from_state_dict(state_dict)
84.         use_cde = True if inferred is None else inferred
85.
86.     model = CondorBrain(
87.         d_model=d_model,
88.         n_layers=n_layers,
89.         input_dim=input_dim,
90.         use_cde=bool(use_cde),
91.         use_topk_moe=bool(config.get("use_topk_moe", config.get("use_topk", False))),
92.     )
93.
94.     state_dict = ckpt["state_dict"] if "state_dict" in ckpt else ckpt
95.     if any(k.startswith("module.") for k in state_dict.keys()):
96.         state_dict = {k.replace("module.", ""): v for k, v in state_dict.items()}
97.
98.     model.load_state_dict(state_dict, strict=True)
99.     model.to(device)
100.    model.eval()
101.
102.    n_params = sum(p.numel() for p in model.parameters())
103.    info = ModelInfo(
104.        name=os.path.basename(ckpt_path).replace(".pth", ""),
105.        ckpt_path=ckpt_path,
106.        seq_len=seq_len,
107.        use_cde=bool(use_cde),
108.        d_model=d_model,
109.        n_layers=n_layers,
110.        n_params=n_params,
111.    )
112.    return ModelAdapter(model, info, device), ckpt
113.
```

5.2 intelligence/audit_cde_vs_mamba2_comparison.py (streaming + T4-safe metrics)

```
1. # intelligence/audit_cde_vs_mamba2_comparison.py
2.
3. import os
4. import argparse
5. import math
6. import json
7. from datetime import datetime
8. from typing import Dict, Any, List, Tuple, Optional
9.
10. import numpy as np
11. import pandas as pd
12. import torch
13. from tqdm import tqdm
14. from sklearn.tree import DecisionTreeRegressor, export_text
15. from sklearn.feature_selection import mutual_info_regression
16. from scipy.spatial.distance import cosine, jensenshannon
17. from scipy.stats import wasserstein_distance, spearmanr
18.
19. from intelligence.canonical_feature_registry import FEATURE_COLS_V22, INPUT_DIM_V22
20. from intelligence.model_adapter import load_model_any
21.
22. DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
23. EPS = 1e-6
24.
25. HEAD_NAMES = [
26.     "call_offset", "put_offset", "wing_width", "dte",
27.     "pop", "roi", "max_loss", "confidence", "entry", "exit"
28. ]
29. ROI_HEAD = 5 # roi
30.
31. # -----
32. # Data: streaming window sampler
33. # -----
34.
35. def stream_sample_windows_csv(
36.     csv_path: str,
37.     feature_cols: List[str],
38.     seq_len: int,
39.     n_samples: int,
40.     seed: int,
41.     max_rows: int = 0,
42.     dtype=np.float32,
43. ) -> np.ndarray:
44.     """
45.     One-pass streaming sampler that returns windows: (n_samples, seq_len, D)
46.     - Does NOT load full CSV into memory.
47.     - Uses a rolling buffer and reservoir sampling over valid window start indices.
48.     """
49.     rng = np.random.default_rng(seed)
50.     D = len(feature_cols)
51.
52.     # Rolling buffer of last seq_len rows (feature-only)
53.     buf = np.zeros((seq_len, D), dtype=dtype)
54.     buf_count = 0
55.
56.     # Reservoir of sampled windows
57.     windows = np.zeros((n_samples, seq_len, D), dtype=dtype)
58.     seen_windows = 0 # number of candidate windows encountered
59.
60.     # Stream CSV in chunks, only required cols, float32
```

```

61.     usecols = feature_cols
62.     dtypes = {c: "float32" for c in usecols}
63.
64.     rows_read = 0
65.     for chunk in pd.read_csv(csv_path, usecols=usecols, dtype=dtypes, chunksize=200_000):
66.         Xc = chunk.to_numpy(copy=False) # (m, D) float32
67.         m = Xc.shape[0]
68.
69.         if max_rows and rows_read + m > max_rows:
70.             Xc = Xc[: max(0, max_rows - rows_read)]
71.             m = Xc.shape[0]
72.
73.         for i in range(m):
74.             buf[buf_count % seq_len] = np.nan_to_num(Xc[i], nan=0.0, posinf=0.0, neginf=0.0)
75.             buf_count += 1
76.             rows_read += 1
77.
78.             if buf_count >= seq_len:
79.                 # materialize window in correct order
80.                 start = (buf_count - seq_len) % seq_len
81.                 if start == 0:
82.                     w = buf.copy()
83.                 else:
84.                     w = np.vstack([buf[start:], buf[:start]])
85.
86.                 # reservoir sample windows uniformly over all seen windows
87.                 if seen_windows < n_samples:
88.                     windows[seen_windows] = w
89.                 else:
90.                     j = rng.integers(0, seen_windows + 1)
91.                     if j < n_samples:
92.                         windows[j] = w
93.
94.                 seen_windows += 1
95.
96.             if max_rows and rows_read >= max_rows:
97.                 break
98.
99.         if max_rows and rows_read >= max_rows:
100.            break
101.
102.     if seen_windows == 0:
103.         raise RuntimeError("No windows could be formed (dataset shorter than seq_len?)")
104.
105.     # If dataset had fewer windows than n_samples, shrink output
106.     if seen_windows < n_samples:
107.         windows = windows[:seen_windows]
108.
109.     return windows
110.
111. # -----
112. # Forward helpers
113. # -----
114.
115. def batch_predict(adapter, X: np.ndarray, batch_size: int, amp_bf16: bool = True) ->
Tuple[np.ndarray, Optional[np.ndarray]]:
116.     """
117.     Returns:
118.         y: (N, H)
119.         pred_logits: (N, K) or None
120.     """
121.     N = X.shape[0]
122.     ys = []
123.     ps = []
124.

```

```

125.     for s in range(0, N, batch_size):
126.         xb = torch.from_numpy(X[s:s+batch_size]).to(DEVICE, non_blocking=True)
127.         with torch.cuda.amp.autocast(enabled=(amp_bf16 and DEVICE.type == "cuda"),
dtype=torch.bfloat16):
128.             out = adapter.predict(xb)
129.             y = out["y"]
130.             pred = out["pred_logits"]
131.
132.             ys.append(y.detach().float().cpu().numpy())
133.             if pred is not None:
134.                 ps.append(pred.detach().float().cpu().numpy())
135.
136.         y_all = np.concatenate(ys, axis=0)
137.         pred_all = np.concatenate(ps, axis=0) if len(ps) else None
138.         return y_all, pred_all
139.
140. # -----
141. # Metrics: permutation importance (batched, CPU shuffle)
142. # -----
143.
144. def permutation_importance(
145.     adapter,
146.     X: np.ndarray,
147.     batch_size: int,
148.     head_index: int = ROI_HEAD,
149.     seed: int = 0,
150. ) -> Dict[str, float]:
151.     """
152.     Computes permutation importance on ROI head only:
153.     I_j = mean_i |y_i - y_i^{perm(j)}|
154.     Memory-safe: does not clone massive GPU tensors per feature.
155.     """
156.     rng = np.random.default_rng(seed)
157.     N, T, D = X.shape
158.
159.     y_base, _ = batch_predict(adapter, X, batch_size=batch_size)
160.     y0 = y_base[:, head_index]
161.
162.     importances = {}
163.     for j in tqdm(range(D), desc=f"Permutation ({adapter.info.name})", leave=False):
164.         Xp = X.copy() # CPU copy once per feature (N*T float32) - OK for N~2000
165.         perm = rng.permutation(N)
166.         Xp[:, :, j] = Xp[perm, :, j]
167.
168.         y_perm, _ = batch_predict(adapter, Xp, batch_size=batch_size)
169.         y1 = y_perm[:, head_index]
170.
171.         imp = float(np.mean(np.abs(y1 - y0)))
172.         importances[str(j)] = imp
173.
174.     # Normalize to 100
175.     s = sum(importances.values()) + EPS
176.     for k in list(importances.keys()):
177.         importances[k] = 100.0 * importances[k] / s
178.     return importances
179.
180. # -----
181. # Metrics: surrogate tree (last step)
182. # -----
183.
184. def surrogate_tree(adapter, X: np.ndarray, batch_size: int, head_index: int = ROI_HEAD) ->
Dict[str, Any]:
185.     y, _ = batch_predict(adapter, X, batch_size=batch_size)
186.     t = y[:, head_index]
187.

```

```

188.     x_last = X[:, -1, :] # (N, D)
189.     tree = DecisionTreeRegressor(max_depth=4, random_state=0)
190.     tree.fit(x_last, t)
191.
192.     r2 = tree.score(x_last, t)
193.     rules = export_text(tree, feature_names=FEATURE_COLS_V22)
194.     return {"r2": float(r2), "rules": rules, "feature_importances":
tree.feature_importances_.tolist()}
195.
196. # -----
197. # Metrics: gradient saliency (streaming accumulation)
198. # -----
199.
200. def gradient_saliency_streaming(adapter, X: np.ndarray, batch_size: int, head_index: int =
ROI_HEAD, max_batches: int = 50) -> Dict[str, Any]:
201.     """
202.     Computes mean absolute gradient w.r.t inputs, without storing raw_grads.
203.     """
204.     N, T, D = X.shape
205.     sal = np.zeros((D,), dtype=np.float64)
206.     t_sal = np.zeros((T,), dtype=np.float64)
207.
208.     batches = 0
209.     for s in tqdm(range(0, N, batch_size), desc=f"GradSal ({adapter.info.name})", leave=False):
210.         xb_np = X[s:s+batch_size]
211.         xb = torch.from_numpy(xb_np).to(DEVICE)
212.         xb.requires_grad_(True)
213.
214.         out = adapter.model(xb)
215.         if isinstance(out, tuple):
216.             out = out[0]
217.         roi = out[:, head_index].sum()
218.
219.         adapter.model.zero_grad(set_to_none=True)
220.         roi.backward()
221.
222.         g = xb.grad.detach().float().abs().mean(dim=0).cpu().numpy() # (T, D) averaged over
batch
223.         sal += g.mean(axis=0)
224.         t_sal += g.mean(axis=1)
225.
226.         xb.requires_grad_(False)
227.         batches += 1
228.         if batches >= max_batches:
229.             break
230.
231.     denom = max(1, batches)
232.     sal = (sal / denom).astype(np.float32)
233.     t_sal = (t_sal / denom).astype(np.float32)
234.     return {"feature_saliency": sal.tolist(), "temporal_saliency": t_sal.tolist()}
235.
236. # -----
237. # Metrics: Fisher (layerwise scalar only)
238. # -----
239.
240. def fisher_layerwise(adapter, X: np.ndarray, batch_size: int, max_batches: int = 50) ->
Dict[str, float]:
241.     """
242.     Layerwise Fisher proxy:
243.     F_layer = mean over batches of sum_{p in layer} (dL/dtheta_p)^2
244.     Avoids allocating fisher_diag tensors (which duplicates model weights in memory).
245.     """
246.     layer_F: Dict[str, float] = {}
247.     batches = 0
248.

```

```

249.     for s in tqdm(range(0, X.shape[0], batch_size), desc=f"Fisher ({adapter.info.name})",
leave=False):
250.         xb = torch.from_numpy(X[s:s+batch_size]).to(DEVICE)
251.
252.         out = adapter.model(xb)
253.         if isinstance(out, tuple):
254.             out = out[0]
255.
256.         # log-likelihood proxy (same spirit as the existing script)
257.         L = -((out - out.mean(dim=0, keepdim=True)) ** 2).sum()
258.
259.         adapter.model.zero_grad(set_to_none=True)
260.         L.backward()
261.
262.         for name, p in adapter.model.named_parameters():
263.             if p.grad is None:
264.                 continue
265.             layer = name.split(".")[0]
266.             v = float(p.grad.detach().float().pow(2).sum().item())
267.             layer_F[layer] = layer_F.get(layer, 0.0) + v
268.
269.         batches += 1
270.         if batches >= max_batches:
271.             break
272.
273.         # normalize
274.         denom = max(1, batches)
275.         for k in list(layer_F.keys()):
276.             layer_F[k] /= denom
277.         return layer_F
278.
279. # -----
280. # Predicate alignment
281. # -----
282.
283. def calibrate_predicates(pred_logits: np.ndarray, tau: float = 1.0) -> np.ndarray:
284.     """
285.     p = sigmoid((1 - median(l))/tau)
286.     """
287.     med = np.median(pred_logits, axis=0, keepdims=True)
288.     z = (pred_logits - med) / max(tau, EPS)
289.     p = 1.0 / (1.0 + np.exp(-z))
290.     return p.astype(np.float32)
291.
292. def predicate_distribution(p: np.ndarray) -> np.ndarray:
293.     m = p.mean(axis=0)
294.     P = (m + EPS) / (m.sum() + EPS)
295.     return P
296.
297. # -----
298. # Divergence metrics
299. # -----
300.
301. def vector_divergences(a: np.ndarray, b: np.ndarray) -> Dict[str, float]:
302.     a = np.asarray(a, dtype=np.float64)
303.     b = np.asarray(b, dtype=np.float64)
304.
305.     # normalize to distributions for JSD/Wasserstein where meaningful
306.     ap = (a - a.min()) + EPS
307.     bp = (b - b.min()) + EPS
308.     ap = ap / ap.sum()
309.     bp = bp / bp.sum()
310.
311.     return {
312.         "cosine": float(1.0 - cosine(a, b)),

```

```

313.         "jsd": float(jensenshannon(ap, bp) ** 2),
314.         "wasserstein": float(wasserstein_distance(ap, bp)),
315.         "spearman_r": float(spearmanr(a, b).correlation),
316.     }
317.
318. # -----
319. # Main
320. # -----
321.
322. def main():
323.     parser = argparse.ArgumentParser()
324.     parser.add_argument("--models", nargs=2, required=True, help="two checkpoints: CDE and Mamba2 (any order)")
325.     parser.add_argument("--data", required=True, help="CSV path")
326.     parser.add_argument("--samples", type=int, default=2000)
327.     parser.add_argument("--seed", type=int, default=42)
328.     parser.add_argument("--batch-size", type=int, default=64)
329.     parser.add_argument("--max-rows", type=int, default=0, help="debug cap on rows streamed")
330.     parser.add_argument("--output", default="reports/cde_vs_mamba2_audit.json")
331.     parser.add_argument("--skip-gradients", action="store_true")
332.     parser.add_argument("--skip-fisher", action="store_true")
333.     parser.add_argument("--skip-permutation", action="store_true")
334.     args = parser.parse_args()
335.
336.     # Load models (architecture-aware)
337.     adapters = []
338.     for p in args.models:
339.         ad, ckpt = load_model_any(p, device=DEVICE, input_dim=INPUT_DIM_V22)
340.         adapters.append(ad)
341.
342.     # Common seq_len
343.     T_common = min(a.info.seq_len for a in adapters)
344.
345.     # Stream sample windows once (shared for both models)
346.     X = stream_sample_windows_csv(
347.         csv_path=args.data,
348.         feature_cols=FEATURE_COLS_V22,
349.         seq_len=T_common,
350.         n_samples=args.samples,
351.         seed=args.seed,
352.         max_rows=args.max_rows,
353.         dtype=np.float32,
354.     )
355.
356.     results: Dict[str, Any] = {
357.         "generated": datetime.now().isoformat(),
358.         "device": str(DEVICE),
359.         "T_common": int(T_common),
360.         "n_windows": int(X.shape[0]),
361.         "models": {},
362.         "pairwise": {},
363.     }
364.
365.     # Per-model analyses
366.     for ad in adapters:
367.         entry: Dict[str, Any] = {
368.             "info": ad.info.__dict__,
369.         }
370.
371.         y, pred = batch_predict(ad, X, batch_size=args.batch_size)
372.         entry["output_mean"] = y.mean(axis=0).tolist()
373.         entry["output_std"] = y.std(axis=0).tolist()
374.
375.         entry["surrogate_tree"] = surrogate_tree(ad, X, batch_size=args.batch_size)
376.

```

```

377.         if not args.skip_permutation:
378.             entry["perm_importance_roi"] = permutation_importance(
379.                 ad, X, batch_size=args.batch_size, head_index=ROI_HEAD, seed=args.seed
380.             )
381.
382.         if not args.skip_gradients:
383.             entry["gradient_saliency"] = gradient_saliency_streaming(
384.                 ad, X, batch_size=max(1, args.batch_size // 4), head_index=ROI_HEAD
385.             )
386.
387.         if not args.skip_fisher:
388.             entry["fisher_layerwise"] = fisher_layerwise(
389.                 ad, X, batch_size=max(1, args.batch_size // 4)
390.             )
391.
392.         # predicates
393.         if pred is not None:
394.             pcal = calibrate_predicates(pred, tau=1.0)
395.             entry["predicate_prob_mean"] = pcal.mean(axis=0).tolist()
396.             entry["predicate_dist"] = predicate_distribution(pcal).tolist()
397.         else:
398.             entry["predicate_prob_mean"] = None
399.             entry["predicate_dist"] = None
400.
401.         results["models"][ad.info.name] = entry
402.
403.         # Pairwise divergences on ROI permutation, gradient, fisher, outputs, predicate dist
404.         a0, a1 = adapters[0].info.name, adapters[1].info.name
405.
406.         # outputs divergence on mean vector
407.         m0 = np.array(results["models"][a0]["output_mean"])
408.         m1 = np.array(results["models"][a1]["output_mean"])
409.         results["pairwise"]["output_mean_div"] = vector_divergences(m0, m1)
410.
411.         # permutation divergence
412.         if "perm_importance_roi" in results["models"][a0] and "perm_importance_roi" in
results["models"][a1]:
413.             p0 = np.array(list(results["models"][a0]["perm_importance_roi"].values()))
414.             p1 = np.array(list(results["models"][a1]["perm_importance_roi"].values()))
415.             results["pairwise"]["perm_div"] = vector_divergences(p0, p1)
416.
417.         # predicate divergence
418.         d0 = results["models"][a0]["predicate_dist"]
419.         d1 = results["models"][a1]["predicate_dist"]
420.         if d0 is not None and d1 is not None:
421.             results["pairwise"]["predicate_jsd"] = float(jensenshannon(d0, d1))
422.

```

```

1.     os.makedirs(os.path.dirname(args.output) or ".", exist_ok=True)
2.     with open(args.output, "w") as f:
3.         json.dump(results, f, indent=2)
4.
5.     print(f"[OK] Wrote audit: {args.output}")
6.
7. if __name__ == "__main__":
8.     main()
9.

```

6) How this solves our specific failure modes

6.1 It never loads the full CSV

- It replaces full `pd.read_csv`

`audit_cde_comparison`

with a streaming sampler that only outputs N_{samples} windows.

6.2 It never allocates per-parameter Fisher buffers

- It replaces `fisher_diag = {name: zeros_like(param) ...}`

MULTI-MODEL NEURAL CDE INTERPRE...

with **layerwise scalar accumulation**.

6.3 It never stores `raw_grads`

- It replaces raw storage

MULTI-MODEL NEURAL CDE INTERPRE...

with streaming accumulation.

6.4 It loads either checkpoint correctly

- It fixes the CDE-only construction (`use_cde=True`)

MULTI-MODEL NEURAL CDE INTERPRE...

by reading config or inferring architecture from weights.

7) Recommended run commands (Lightning AI)

Start conservative:

```
python intelligence/audit_cde_vs_mamba2_comparison.py \  
--models models/epoch_best_020226.pth models/Mamba2_0.6477_e1_best.pth \  
--data data/processed/mamba_institutional_2025_lm_v22.csv \  
--samples 2000 \  
--batch-size 32 \  
--skip-gradients \  
--skip-fisher \  
--output reports/cde_vs_mamba2_minimal.json
```

Then add modules one at a time:

```
# add gradients
python intelligence/audit_cde_vs_mamba2_comparison.py ... --batch-size 16 --
output reports/cde_vs_mamba2_grad.json

# add fisher (still safe)
python intelligence/audit_cde_vs_mamba2_comparison.py ... --batch-size 8 --
output reports/cde_vs_mamba2_fisher.json
```

8) Optional upgrades (I definitely want “physics” parity)

We definitely want *true apples-to-apples* “dynamics” metrics:

8.1 Attribute to increments ΔX_t for both

Define:

$$\Delta X_t = X_t - X_{t-1}$$

- For CDE, this matches the discretized intuition.
- For Mamba2, it normalizes attribution to *changes*, not levels.

Implementation: compute gradients w.r.t. $\mathbf{dx}$ by feeding $\mathbf{x}$ but post-multiply saliency by $|\mathbf{dx}|$ or explicitly build $\mathbf{dx}$ as input to an input-projection wrapper.

8.2 Compare stability via Lipschitz proxies

A practical proxy on sampled windows:

$$L \approx \mathbb{E} \left[\frac{\| f(X + \delta) - f(X) \|_2}{\| \delta \|_2 + \varepsilon} \right]$$

We already have a sensitivity/Lyapunov style foundation in my original spec

MULTI-MODEL NEURAL CDE INTERPRE...

; just apply it to both models with the same perturbations δ and the same ROI head.

9) Notes on inequality learning integration

The training pipeline can export discovered predicates into checkpoints

`train_condor_brain`

, but the audit must be able to *read them* and/or *extract their activations*. If the Mamba2 run uses the same predicate modules, the adapter will pick up candidate (B, K) tensors from tuple outputs. If not, predicate comparison will be `None` and the report will still be valid (it'll become “head + attribution” only).

We need and want predicate outputs guaranteed, so the cleanest follow-up is:

Add an explicit `return_predicates=True` flag in `CondorBrain.forward(...)` to return predicate logits in a named position, rather than guessing from tuple contents.

10) What I suggest we modify in the original

`audit_cde_comparison.py` (surgical patch list)

1. Replace full CSV load

`audit_cde_comparison`

with streaming sampler.

2. Replace `load_cde_model` (hard-coded `use_cde=True`)

MULTI-MODEL NEURAL CDE INTERPRE...

with architecture-aware loader.

3. Remove raw gradient storage

MULTI-MODEL NEURAL CDE INTERPRE...

4. Replace Fisher tensor buffers

MULTI-MODEL NEURAL CDE INTERPRE...

with layer wise scalar Fisher.

5. Keep Hessian skipped on T4; if needed, implement **HVP-only** with a *tiny* random subspace.